

```
%pip install -q "gradio>=5.0,<7" "timm==1.0.19" "scikit-learn==1.7.1" "pandas==2.2.3" "Pillow==11.3.0" "joblib>=1.4"
```

```
from google.colab import drive

drive.mount("/content/drive")

import hashlib
import json
import shutil
from pathlib import Path

import gradio as gr
import joblib
import matplotlib
import numpy as np
import pandas as pd
import timm
import torch
import torch.nn.functional as F
from PIL import Image, ImageOps
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve
from torchvision import transforms

PROJECT_ROOT = Path("/content/drive/MyDrive/isic_label_efficiency_v1")
RESULT_DIR = PROJECT_ROOT / "results_v10_final_a"
FEATURE_DIR = PROJECT_ROOT / "features"
MODEL_DIR = PROJECT_ROOT / "models"
MANIFEST_DIR = PROJECT_ROOT / "manifests"
DEMO_DIR = PROJECT_ROOT / "v10_clinical_demo_assets"

CHECKPOINT_PATH = MODEL_DIR / "dinov2_isic_domain_adapted_v2.pt"
SPLIT_MANIFEST_PATH = MANIFEST_DIR / "patient_split_seed_42.csv"
PROBE_BUNDLE_PATH = MODEL_DIR / "dinov2_domain_probe_25pct_seed0_v1.joblib"

MODEL_KEY = "dinov2_domain"
MODEL_NAME = "vit_small_patch14_reg4_dinov2.lvd142m"
IMAGE_SIZE = 224
LABEL_FRACTION = 0.25
BUDGET_SEED = 0
TARGET_SENSITIVITY = 0.80
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Device:", DEVICE)
print("Project:", PROJECT_ROOT)
print("Results:", RESULT_DIR)
```

```
Mounted at /content/drive
Device: cpu
Project: /content/drive/MyDrive/isic_label_efficiency_v1
Results: /content/drive/MyDrive/isic_label_efficiency_v1/results_v10_final_a
```

```
REQUIRED_RESULT_FILES = [
    "dino_checkpoint_provenance.csv",
    "explainability_configuration.csv",
    "explainability_cases.csv",
    "v10_run_manifest.json",
    "validation_label_efficiency.png",
    "validation_gradcam_comparison_extreme.png",
    "validation_occlusion_extreme.png",
```

```

]

missing = [
    name for name in REQUIRED_RESULT_FILES
    if not (RESULT_DIR / name).is_file()
]
assert not missing, f"Missing V10 result files: {missing}"
assert CHECKPOINT_PATH.is_file(), f"Missing checkpoint: {CHECKPOINT_PATH}"
assert SPLIT_MANIFEST_PATH.is_file(), (
    f"Missing frozen split manifest: {SPLIT_MANIFEST_PATH}"
)

train_cache_candidates = list(
    FEATURE_DIR.glob("dinov2_domain__probe_pool__*.npz")
)
validation_cache_candidates = list(
    FEATURE_DIR.glob("dinov2_domain__validation__*.npz")
)
assert train_cache_candidates, "No domain-DINO probe-pool cache was found."
assert validation_cache_candidates, "No domain-DINO validation cache was found."

# V10 migration can leave a legacy and a current cache. The current copy
# is the newest; a replay check below verifies that it matches V10 scores.
TRAIN_CACHE_PATH = max(
    train_cache_candidates, key=lambda path: path.stat().st_mtime
)
VALIDATION_CACHE_PATH = max(
    validation_cache_candidates, key=lambda path: path.stat().st_mtime
)

def file_sha256(path, chunk_size=1024 * 1024):
    digest = hashlib.sha256()
    with Path(path).open("rb") as handle:
        for chunk in iter(lambda: handle.read(chunk_size), b""):
            digest.update(chunk)
    return digest.hexdigest()

provenance = pd.read_csv(RESULT_DIR / "dino_checkpoint_provenance.csv")
expected_sha256 = str(provenance.iloc[0]["sha256"])
actual_sha256 = file_sha256(CHECKPOINT_PATH)
assert actual_sha256 == expected_sha256, (
    "Checkpoint checksum does not match the completed V10 run."
)

DISPLAY_ASSETS = [
    "validation_label_efficiency.png",
    "validation_gradcam_comparison_extreme.png",
    "validation_occlusion_extreme.png",
]
DEMO_DIR.mkdir(parents=True, exist_ok=True)
for name in DISPLAY_ASSETS:
    shutil.copy2(RESULT_DIR / name, DEMO_DIR / name)

print("Checkpoint verified:", actual_sha256)
print("Training cache:", TRAIN_CACHE_PATH.name)
print("Validation cache:", VALIDATION_CACHE_PATH.name)
print("Demo-safe figures:", len(DISPLAY_ASSETS))

```

```

Checkpoint verified: bf9e00858ff80f1073db6be9129bfc45b03a9a3ede67b7b5857f5b2f56cc52e6
Training cache: dinov2_domain__probe_pool__96cad35ddb93922d.npz
Validation cache: dinov2_domain__validation__e02e3b0a2fe64ca6.npz
Demo-safe figures: 3

```

```
# Step 4
```

```

def l2_normalize(features):
    features = np.asarray(features, dtype=np.float32)
    return features / (
        np.linalg.norm(features, axis=1, keepdims=True) + 1e-8
    )

def read_feature_cache(path):
    with np.load(path, allow_pickle=False) as cached:
        return {
            "X": cached["X"].astype(np.float32),
            "y": cached["y"].astype(np.int64),
            "isic_id": cached["isic_id"].astype(str),
            "patient_id": cached["patient_id"].astype(str),
        }

```

```

def select_budget_patients(patient_ids, fraction, seed):
    # V10 selected the patient budget from train_pool before applying
    # the compute cap. The frozen split manifest preserves train_pool's
    # row order, so duplicates must be removed without sorting first.
    patients = pd.Series(patient_ids).drop_duplicates().to_numpy()
    n_patients = max(2, int(round(len(patients) * fraction)))
    rng = np.random.default_rng(seed)
    return np.sort(
        rng.choice(patients, size=n_patients, replace=False)
    )

probe_configuration = pd.read_csv(
    RESULT_DIR / "explainability_configuration.csv"
)
config_row = probe_configuration[
    (probe_configuration["backbone"] == MODEL_KEY)
    & np.isclose(
        probe_configuration["label_fraction"], LABEL_FRACTION
    )
    & (probe_configuration["seed"] == BUDGET_SEED)
]
assert len(config_row) == 1, "Expected exactly one matching V10 probe configuration."
selected_C = float(config_row.iloc[0]["C"])

bundle_is_current = False
if PROBE_BUNDLE_PATH.is_file():
    candidate_bundle = joblib.load(PROBE_BUNDLE_PATH)
    bundle_is_current = (
        candidate_bundle.get("checkpoint_sha256") == expected_sha256
        and candidate_bundle.get("model_key") == MODEL_KEY
        and np.isclose(
            candidate_bundle.get("label_fraction", -1), LABEL_FRACTION
        )
        and candidate_bundle.get("budget_seed") == BUDGET_SEED
        and np.isclose(candidate_bundle.get("C", -1), selected_C)
    )

if bundle_is_current:
    probe_bundle = candidate_bundle
    probe = probe_bundle["probe"]
    print("Loaded the existing verified probe bundle:", PROBE_BUNDLE_PATH)
else:
    train_cache = read_feature_cache(TRAIN_CACHE_PATH)
    split_manifest = pd.read_csv(
        SPLIT_MANIFEST_PATH,
        usecols=["patient_id", "split"],
        dtype={"patient_id": str, "split": str},
    )
    train_pool_patient_ids = split_manifest.loc[
        split_manifest["split"] == "train", "patient_id"
    ].to_numpy()
    chosen_patients = select_budget_patients(
        train_pool_patient_ids, LABEL_FRACTION, BUDGET_SEED
    )
    train_mask = np.isin(
        train_cache["patient_id"], chosen_patients
    )
    X_budget = l2_normalize(train_cache["X"][train_mask])
    y_budget = train_cache["y"][train_mask]
    assert np.unique(y_budget).size == 2, "Selected budget has only one class."

    probe = LogisticRegression(
        C=selected_C,
        max_iter=3000,
        class_weight="balanced",
        solver="lbfgs",
        random_state=BUDGET_SEED,
    )
    probe.fit(X_budget, y_budget)

    validation_cache = read_feature_cache(VALIDATION_CACHE_PATH)
    validation_scores = probe.predict_proba(
        l2_normalize(validation_cache["X"])
    )[:, 1]

    # Replay the saved V10 scores for the nine explanation cases.
    saved_cases = pd.read_csv(
        RESULT_DIR / "explainability_cases.csv",
        dtype={"isic_id": str},

```

```

    )
    score_lookup = dict(zip(
        validation_cache["isic_id"], validation_scores
    ))
    replayed = np.asarray([
        score_lookup[isic_id]
        for isic_id in saved_cases["isic_id"]
    ])
    expected = saved_cases["score_dinov2_domain"].to_numpy()
    max_replay_difference = float(
        np.max(np.abs(replayed - expected))
    )
    assert max_replay_difference < 2e-5, (
        "Probe replay did not match V10. Do not launch the demo. "
        f"Maximum difference: {max_replay_difference:.6g}"
    )

    # Exploratory operating point: among ROC points reaching at least
    # 80% sensitivity, choose the point with the lowest false-positive rate.
    fpr, tpr, thresholds = roc_curve(
        validation_cache["y"], validation_scores
    )
    eligible = np.flatnonzero(tpr >= TARGET_SENSITIVITY)
    assert len(eligible), "No validation threshold reached the target sensitivity."
    best = eligible[np.argmin(fpr[eligible])]
    review_threshold = float(thresholds[best])
    validation_sensitivity = float(tpr[best])
    validation_specificity = float(1.0 - fpr[best])

    probe_bundle = {
        "probe": probe,
        "model_key": MODEL_KEY,
        "label_fraction": LABEL_FRACTION,
        "budget_seed": BUDGET_SEED,
        "C": selected_C,
        "checkpoint_sha256": expected_sha256,
        "review_threshold": review_threshold,
        "validation_sensitivity": validation_sensitivity,
        "validation_specificity": validation_specificity,
        "threshold_source": (
            "validation ROC; minimum target sensitivity 0.80; exploratory only"
        ),
        "score_is_calibrated_probability": False,
        "max_case_replay_difference": max_replay_difference,
    }
    joblib.dump(probe_bundle, PROBE_BUNDLE_PATH)
    print("Reconstructed, verified, and saved:", PROBE_BUNDLE_PATH)

    review_threshold = float(probe_bundle["review_threshold"])
    validation_sensitivity = float(
        probe_bundle["validation_sensitivity"]
    )
    validation_specificity = float(
        probe_bundle["validation_specificity"]
    )

    assert np.array_equal(probe.classes_, np.array([0, 1]))
    print(f"Probe C: {selected_C:g}")
    print(f"Exploratory review threshold: {review_threshold:.4f}")
    print(
        "Validation-only operating point: "
        f"sensitivity={validation_sensitivity:.3f}, "
        f"specificity={validation_specificity:.3f}"
    )
    print("The score is uncalibrated and is NOT a probability of cancer.")

```

Loaded the existing verified probe bundle: /content/drive/MyDrive/isic_label_efficiency_v1/models/dinov2_domain_probe_25pct_

Probe C: 1

Exploratory review threshold: 0.2920

Validation-only operating point: sensitivity=0.806, specificity=0.804

The score is uncalibrated and is NOT a probability of cancer.

step 6

```

class LinearProbeGradCAM:
    def __init__(self, model, classifier, target_layer, device):
        self.model = model
        self.classifier = classifier
        self.device = device
        self.activations = None
        self.gradients = None

```

```

self.weight = torch.as_tensor(
    classifier.coef_[0],
    dtype=torch.float32,
    device=device,
)
self.bias = torch.as_tensor(
    classifier.intercept_[0],
    dtype=torch.float32,
    device=device,
)
self.hook = target_layer.register_forward_hook(
    self._capture_activations
)

def _capture_activations(self, module, inputs, output):
    if isinstance(output, (tuple, list)):
        output = output[0]
    self.activations = output
    output.register_hook(self._capture_gradients)

def _capture_gradients(self, gradient):
    self.gradients = gradient

def _to_grid(self, tensor):
    prefix_tokens = int(
        getattr(self.model, "num_prefix_tokens", 1)
    )
    patch_tokens = tensor[:, prefix_tokens:, :]
    grid_height, grid_width = map(
        int, self.model.patch_embed.grid_size
    )
    assert patch_tokens.shape[1] == grid_height * grid_width
    return (
        patch_tokens.reshape(
            patch_tokens.shape[0],
            grid_height,
            grid_width,
            patch_tokens.shape[2],
        )
        .permute(0, 3, 1, 2)
        .contiguous()
    )

def __call__(self, model_input):
    images = model_input.unsqueeze(0).to(
        self.device, dtype=torch.float32
    )
    images.requires_grad_(True)
    self.model.zero_grad(set_to_none=True)
    self.activations = None
    self.gradients = None

    with torch.enable_grad():
        features = self.model(images)
        normalized = F.normalize(
            features, p=2, dim=1, eps=1e-8
        )
        logit = normalized @ self.weight + self.bias
        logit.sum().backward()

    activations = self._to_grid(self.activations)
    gradients = self._to_grid(self.gradients)
    channel_weights = gradients.mean(
        dim=(2, 3), keepdim=True
    )
    raw_cam = torch.relu(
        (channel_weights * activations).sum(dim=1)
    )
    raw_cam = F.interpolate(
        raw_cam.unsqueeze(1),
        size=(IMAGE_SIZE, IMAGE_SIZE),
        mode="bilinear",
        align_corners=False,
    ).squeeze(1)
    cam_min = raw_cam.amin(dim=(1, 2), keepdim=True)
    cam_max = raw_cam.amax(dim=(1, 2), keepdim=True)
    cam = (raw_cam - cam_min) / (
        cam_max - cam_min
    ).clamp_min(1e-8)
    score = torch.sigmoid(logit)
    return (
        cam[0].detach().cpu().numpy(),

```

```

        float(score[0].detach().cpu()),
    )

# Initialize the backbone model
backbone = timm.create_model(MODEL_NAME, pretrained=False, img_size=IMAGE_SIZE)
backbone.load_state_dict(torch.load(CHECKPOINT_PATH, map_location="cpu")['state_dict'])
backbone = backbone.to(DEVICE)
backbone.eval()

# Define image transformations
NORMALIZE_MEAN = (0.485, 0.456, 0.406)
NORMALIZE_STD = (0.229, 0.224, 0.225)

eval_transform = transforms.Compose([
    transforms.Resize(IMAGE_SIZE, interpolation=transforms.InterpolationMode.BICUBIC),
    transforms.CenterCrop(IMAGE_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(NORMALIZE_MEAN, NORMALIZE_STD),
])

display_transform = transforms.Compose([
    transforms.Resize(IMAGE_SIZE, interpolation=transforms.InterpolationMode.BICUBIC),
    transforms.CenterCrop(IMAGE_SIZE),
])

gradcam = LinearProbeGradCAM(
    backbone,
    probe,
    backbone.blocks[-2].norm1,
    DEVICE,
)

def prepare_image(image):
    image = ImageOps.exif_transpose(image).convert("RGB")
    display_image = display_transform(image)
    model_input = eval_transform(image)
    return display_image, model_input

def quality_gate(image):
    image = ImageOps.exif_transpose(image).convert("RGB")
    width, height = image.size
    gray = np.asarray(image.convert("L"), dtype=np.float32)
    mean_brightness = float(gray.mean())
    contrast = float(gray.std())

    reasons = []
    warnings = []
    if min(width, height) < IMAGE_SIZE:
        reasons.append(
            f"image is too small ({width}x{height}; minimum side {IMAGE_SIZE}px)"
        )
    if mean_brightness < 15:
        reasons.append("image is extremely dark")
    if mean_brightness > 240:
        reasons.append("image is extremely bright")
    if contrast < 8:
        reasons.append("image has very little visible contrast")
    if max(width, height) / max(1, min(width, height)) > 2.5:
        warnings.append(
            "The image is unusually elongated; the centre crop may omit the lesion."
        )
    return reasons, warnings

@torch.inference_mode()
def predict_score(model_input):
    features = backbone(
        model_input.unsqueeze(0).to(DEVICE)
    )
    normalized = F.normalize(
        features.float(), p=2, dim=1, eps=1e-8
    )
    coefficient = torch.as_tensor(
        probe.coef_[0], dtype=torch.float32, device=DEVICE
    )
    intercept = torch.as_tensor(
        probe.intercept_[0], dtype=torch.float32, device=DEVICE
    )
    score = torch.sigmoid(

```

```

        normalized @ coefficient + intercept
    )
    return float(score[0].cpu())

def make_overlay(display_image, cam):
    image_array = np.asarray(
        display_image,
        dtype=np.float32
    ) / 255.0
    heatmap = matplotlib.colormaps["magma"](cam)[..., :3]
    overlay = np.clip(
        0.62 * image_array + 0.38 * heatmap,
        0.0,
        1.0,
    )
    return Image.fromarray(
        np.uint8(np.round(overlay * 255.0)), mode="RGB"
    )

def assess_image(image, audience, make_explanation, acknowledged):
    if not acknowledged:
        return (
            None,
            "### Acknowledgement required\nPlease confirm that you understand this is a research prototype.",
            "No image was processed.",
        )
    if image is None:
        return None, "### Upload an image first", "No image was processed."

    reasons, warnings = quality_gate(image)
    if reasons:
        reason_text = "; ".join(reasons)
        return (
            None,
            "### Unable to assess\n"
            f"The prototype abstained because the {reason_text}. "
            "Retake the photograph in even lighting with the lesion centred. "
            "Do not delay professional assessment because the app abstained.",
            "***Automated quality gate:** failed. This is not a medical image-quality assessment.",
        )

    display_image, model_input = prepare_image(image)
    if make_explanation:
        cam, score = gradcam(model_input)
        overlay = make_overlay(display_image, cam)
    else:
        score = predict_score(model_input)
        overlay = None

    is_flagged = score >= review_threshold
    warning_text = " ".join(warnings)
    common = (
        f"\n\n**Research score:** {score:.3f} "
        "\nThis uncalibrated score is not a probability of cancer. "
        "The threshold was selected on the validation set and has not been "
        "confirmed on the locked test set or an external clinical population."
    )

    if audience == "Patient / public":
        if is_flagged:
            result = (
                "### Research model flag: clinical review suggested\n"
                "The image produced a higher model signal. This does **not** "
                "mean that the lesion is cancerous. Arrange assessment by a "
                "qualified healthcare professional, particularly if the mark "
                "is new, changing, painful, itchy, inflamed, bleeding, or crusty."
            )
        else:
            result = (
                "### No high model signal detected – cancer is not ruled out\n"
                "This result does **not** mean the lesion is benign or safe. "
                "The model can miss important lesions. Seek professional advice "
                "for any new, changing, or concerning mark regardless of this result."
            )
        else:
            if is_flagged:
                result = (
                    "### Experimental review flag: above validation threshold\n"
                    f"Score {score:.3f} is at or above the exploratory threshold "
                    f"({review_threshold:.3f}). Do not use this output independently "

```

```

        "for triage, diagnosis, referral, or treatment."
    )
    else:
        result = (
            "### Experimental review flag: below validation threshold\n"
            f"Score {score:.3f} is below the exploratory threshold "
            f"({review_threshold:.3f}). This must not be used to rule out "
            "malignancy or to defer normal clinical assessment."
        )

    result += common
    quality = (
        "**Automated quality gate:** passed its limited technical checks. "
        "This does not establish clinical suitability or detect dataset shift."
    )
    if warning_text:
        quality += "\n\n**Warning:** " + warning_text
    if make_explanation:
        quality += (
            "\n\nThe heatmap shows model sensitivity, not a lesion boundary, "
            "causal explanation, or proof of disease."
        )
    return overlay, result, quality

print("Inference functions are ready.")

```

Inference functions are ready.

step 7

```

APP_CSS = '''
.warning-banner {
    background: #fff4e5;
    border: 2px solid #b45309;
    border-radius: 12px;
    color: #5b2c06;
    padding: 14px 16px;
    margin-bottom: 14px;
}
.result-panel {
    border-left: 5px solid #2457a6;
    padding-left: 14px;
}
'''

with gr.Blocks(
    title="Skin Lesion Decision-Support Research Prototype",
    theme=gr.themes.Soft(),
    css=APP_CSS,
) as demo:
    gr.HTML(
        '''
        <div class="warning-banner">
            <strong>Research prototype – not a diagnostic system.</strong><br>
            Do not use this app to diagnose or rule out skin cancer, make a
            treatment decision, or delay professional assessment.
        </div>
        '''
    )
    gr.Markdown(
        """# Skin Lesion Decision-Support Research Prototype\n
        "Upload one centred lesion image. The model returns an experimental "
        "review flag and, optionally, a sensitivity heatmap."
    )

    with gr.Tab("Assess an image"):
        with gr.Row():
            with gr.Column(scale=1):
                image_input = gr.Image(
                    type="pil",
                    sources=["upload", "webcam", "clipboard"],
                    label="Skin-lesion image",
                )
            audience = gr.Radio(
                ["Clinician / researcher", "Patient / public"],
                value="Clinician / researcher",
                label="Who is viewing the demonstration?",
            )
            explanation_toggle = gr.Checkbox(
                value=True,

```

```

        label="Generate Grad-CAM sensitivity heatmap",
        info="Turn this off for faster CPU inference.",
    )
    acknowledgement = gr.Checkbox(
        value=False,
        label=(
            "I understand this is research-only and is not a diagnosis."
        ),
    )
    assess_button = gr.Button(
        "Run research assessment",
        variant="primary",
    )
    clear_button = gr.ClearButton(
        [image_input, acknowledgement]
    )

    with gr.Column(scale=1):
        result_output = gr.Markdown(
            "### No image assessed",
            elem_classes=["result-panel"],
        )
        heatmap_output = gr.Image(
            label="Grad-CAM sensitivity overlay",
            interactive=False,
        )
        quality_output = gr.Markdown(
            "No image was processed."
        )

    assess_button.click(
        fn=assess_image,
        inputs=[
            image_input,
            audience,
            explanation_toggle,
            acknowledgement,
        ],
        outputs=[
            heatmap_output,
            result_output,
            quality_output,
        ],
    )

    with gr.Tab("Validation evidence"):
        gr.Markdown(
            "These figures describe the completed validation experiment. "
            "They are not evidence of clinical deployment performance."
        )
        gr.Image(
            value=str(
                DEMO_DIR / "validation_label_efficiency.png"
            ),
            label="Validation label-efficiency results",
            interactive=False,
        )
        with gr.Row():
            gr.Image(
                value=str(
                    DEMO_DIR
                    / "validation_gradcam_comparison_extreme.png"
                ),
                label="Precomputed Grad-CAM examples",
                interactive=False,
            )
            gr.Image(
                value=str(
                    DEMO_DIR
                    / "validation_occlusion_extreme.png"
                ),
                label="Precomputed occlusion examples",
                interactive=False,
            )

    with gr.Tab("Intended use and limitations"):
        gr.Markdown(
            f'''
            ### Intended demonstration use

            - Educational demonstration of a research imaging workflow.
            - One image at a time; no patient record or longitudinal comparison.
        '''
    )

```

- Model: domain-adapted DINOv2 with a linear probe fitted using a simulated {LABEL_FRACTION:.0%} patient-label

Validation-only operating point

- Review threshold: **{review_threshold:.3f}**
- Observed validation sensitivity: **{validation_sensitivity:.3f}**
- Observed validation specificity: **{validation_specificity:.3f}**
- These values are exploratory and are not expected to transfer unchanged to real clinical use.

Critical limitations

- The locked test set was not scored in V10.
- No external, prospective, subgroup, calibration, or clinical-impact validation has been completed.
- The input may differ substantially from the ISIC training images, particularly for ordinary phone photographs.
- The score is uncalibrated and is not a cancer probability.
- A low score cannot rule out malignancy.
- Grad-CAM is method- and layer-sensitive and is not a causal explanation.
- This prototype is not a registered or approved medical device.

Official patient information: [NHS melanoma symptoms](https://www.nhs.uk/conditions/melanoma-skin-cancer/symptoms)

```
demo.queue(default_concurrency_limit=1, max_size=8)
print("Gradio interface built successfully.")
```

```
/tmp/ipykernel_3652/2276760986.py:16: UserWarning: The parameters have been moved from the Blocks constructor to the launch()
    with gr.Blocks(
Gradio interface built successfully.
```

step 8

```
from getpass import getpass

DEMO_USERNAME = "reviewer"
DEMO_PASSWORD = getpass("Choose a temporary demo password (8+ characters): ")
assert len(DEMO_PASSWORD) >= 8, "Please use at least 8 characters."

try:
    demo.close()
except Exception:
    pass

demo.launch(
    share=True,
    debug=True,
    auth=(DEMO_USERNAME, DEMO_PASSWORD),
    allowed_paths=[str(DEMO_DIR)],
    show_error=True,
)
```

Choose a temporary demo password (8+ characters):

Colab notebook detected. This cell will run indefinitely so that you can see errors and logs. To turn off, set debug=False i

* Running on public URL: <https://6236b7cdbbedb10d72.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run